

Build or Buy? A Hybrid, Zero-Trust Customer Identity and Access Management Platform for Modern Businesses

Robert C. Oliver, Jr. Travis L. Lester

Dept. of Computer Science, Lewis University, Romeoville, IL, USA

CPSC 69100 Master's Project · Advisor: Dr. Al-Sharif

Abstract—Modern identity systems are the primary target of attackers: stolen credentials are among the most common and slowest-to-contain entry points into an organization, while the commercial Customer Identity and Access Management (CIAM) market is priced per monthly active user and tuned for enterprise scale. This leaves small businesses, startups, schools, and non-profits without an affordable, sovereign option. This paper asks whether an organization should build or buy its identity system, and answers with a working reference implementation. TesseraApp pairs an in-house authentication core—BCrypt credentials, HMAC-SHA-512 JSON Web Tokens, permission-based role access control, authenticator-app multi-factor authentication, discoverable passkey (WebAuthn) sign-in, and server-tracked refresh sessions with rotation and reuse detection—with federated sign-in delegated to Google, GitHub, and Microsoft via OAuth2/OpenID Connect. Every authentication path converges on one token-issuance point, so authorization, MFA policy, and audit logging apply uniformly. We present the project end to end: motivation, a survey of the CIAM landscape, an IEEE-830 requirements specification with context, use-case, and activity models, the 4+1 architecture, the two-phase implementation, and an evaluation with a future-work agenda.

Index Terms—identity and access management, CIAM, zero-trust architecture, OAuth2, OpenID Connect, JSON Web Token, multi-factor authentication, WebAuthn, passkeys, refresh-token rotation, role-based access control, Spring Security, Angular.

I. Introduction

A. The problem. Many organizations rely on third-party identity and access management (IAM) providers—Okta, Auth0, Microsoft, ForgeRock—to centralize authentication and reduce the need for dedicated in-house security teams. These platforms are effective and widely adopted, but they are designed for a broad range of customers rather than the specific operational security requirements of any one company. Organizations can face real limits on customization, flexibility, performance optimization, and—most consequentially—control over sensitive authentication data.

At the same time, a fully isolated in-house system introduces its own friction: users must manage yet another credential, and the organization loses the single sign-on convenience employees already rely on through Google or Microsoft. Neither pole—total vendor dependency nor total self-management—is ideal.

This project designs and implements a hybrid identity sys-

tem that combines an in-house authentication core with federated identity through third-party OAuth2 providers, keeping centralized control over authentication policy, audit logging, and role management while still allowing users to sign in with either in-house credentials or a trusted external provider.

B. Motivation and contributions. The dependence on cloud applications and remote access has made IAM one of the most important areas of cyber security. The practical middle path is to use external providers *only* to verify *who* a user is, then take control of *what* that user may do through an internally managed authorization layer. The principal contribution is a documented reference implementation of a hybrid stateless/stateful authentication model that resolves the scalability-versus-revocability tension in favor of zero-trust's demand for continuous, revocable verification. Concretely we: (1) implement a production-style authentication core (JWT issuance/validation, refresh-session rotation with reuse detection, MFA, passkeys, federation); (2) provide permission-based RBAC with organization scoping; (3) deliver an administrative console and user self-service; (4) secure a real business domain—customers and invoices—behind the identity layer; and (5) record every security-relevant action to a durable audit trail.

C. Where the problem occurs. Three recurring situations motivate the hybrid design. (1) A healthcare or financial-services organization must comply with regulations requiring a full review of authentication events and access-control decisions; a generic third-party vendor may not expose that level of logging, and a naive in-house build may omit the audit trail entirely. A hybrid system keeps a complete internal event log regardless of how a user authenticated. (2) A company wants employees to log in with their existing corporate Google or Microsoft accounts for convenience, but still needs to enforce its own access rules; a hybrid system accepts the external identity while retaining full internal authority over what that identity is permitted to do. (3) A startup building an internal tooling platform lacks the expertise to build authentication correctly from scratch, but cannot justify per-user enterprise licensing; a hybrid approach offloads initial credential verification to a trusted provider while the startup still owns its authorization infrastructure.

D. The complementary-layers insight. OAuth2 and the in-house JWT system operate at different layers and are complementary rather than competing. OAuth2 handles identity verification—who the user is, confirmed by a trusted external provider; the in-house JWT pipeline handles authorization—what that user is allowed to do within the application. The two

paths converge at a token-exchange point: after a successful OAuth2 callback, the system creates or retrieves a local user record and issues its own JWT, so the same RBAC enforcement, MFA policy, and audit logging apply no matter the authentication method. External providers are used *solely* to verify who a user is; after that, the application retains full internal authority over what the user may do.

E. System components. Table I names the three primary components and their responsibilities.

F. Development phases. The build was organized into five phases (Table II), each ending in a working increment.

II. Literature Review — Build vs. Buy

A. Identity is the new perimeter. IAM emerged as a formal discipline in the 1990s with directory tools such as Novell eDirectory and Microsoft Active Directory [21]. The wider IAM market is forecast to reach USD 42.6 billion by 2030, with the CIAM segment alone projected at USD 22.5 billion, up from USD 14.1 billion in 2025 [13]. Security has moved from the network onto identity itself: behind every request is a person, device, or service, treated as “*never trust, always verify*” [1].

IAM and CIAM differ in kind. IAM governs employees—a finite, known population—whereas CIAM governs customers who self-register and consent to data use, at internet scale, where login friction directly affects revenue [12].

B. Why credentials are the target. IBM’s *Cost of a Data Breach Report 2024* put the average breach cost at USD 4.88 million—an all-time high—and named stolen credentials among both the most common and most damaging entry points, taking about 292 days on average to contain [15]. An attacker with valid credentials is not defeating a firewall; they are simply logging in, which places the component deciding how credentials are checked and revoked at the center of an organization’s security.

C. Theoretical foundations. Zero-trust architecture was formalized by NIST in SP 800-207 [1]: no request is trusted by virtue of its network location; each is authenticated and authorized on its own and re-checked continuously. Re-checking trust mid-session implies the ability to *revoke* it mid-session—in tension with the most popular token type [2]. OAuth 2.0 [7] grants an app limited access on a user’s behalf without exposing the password; OpenID Connect adds the identity layer via an ID token and userinfo endpoint [10]. A formal security analysis of OAuth 2.0 uncovered previously unnoticed attacks and then proved the corrected flows secure [3].

Stateless vs. stateful sessions. A JWT packs a session’s claims into a signed token any service can verify alone [8]; this scales well, but the token stays valid until it expires, making early cancellation expensive. An opaque server-side session gives instant revocation at the cost of the shared state JWTs were adopted to avoid [7]. This trade-off is precisely where TesseractApp’s hybrid model is positioned.

Multi-factor. Time-based one-time passwords (RFC 6238) combine current time and a shared secret into a time-limited code, widely supported without per-message infrastructure [9].

Passwordless FIDO2/WebAuthn represents the field’s likely direction, though usability research finds adoption and recovery friction still limit it [6]; TesseractApp implements this device-bound, phishing-resistant approach directly (Section VI-B). Federated-identity security studies characterize the added attack surface of sharing identity across organizations [5], and a systematic review of IAM metrics provides a vocabulary for evaluating identity controls [4].

D. Existing CIAM systems. Products align on a single axis: how much of the identity stack does the organization manage itself? The table summarizes the landscape from managed SaaS to a fully in-house build.

E. Where TesseractApp sits. Keycloak is the pivotal entry in the survey: as the closest commercially supported approximation of a complete in-house build, it demonstrates that a genuine market exists for self-hostable CIAM—customers who require control and data ownership without per-MAU pricing [20].

F. The build-versus-buy decision. TesseractApp lies at the far, self-reliant end of Table III: rather than adopting an open-source platform, it implements the identity stack directly, amplifying Keycloak’s advantages—total control, no per-MAU fees, complete data ownership—at the cost of owning every security property itself [20]. It goes further by being deliberately hybrid rather than all-in-one: stateless JWTs give scalable, infrastructure-free verification, while a stateful trust layer restores per-user device lists, token revocation, and refresh-token rotation with reuse detection. That resolves the scalability-versus-revocability trade-off [8] in favor of zero-trust’s demand for continuous, revocable verification [2], while staying inside the OAuth/OIDC model whose security is formally understood [3]. Authentication remains local and self-managed: credentials are stored as salted BCrypt hashes and access is governed by permission-based RBAC.

A balanced assessment must also state the case *against* building. The access-management market grows at double-digit rates precisely because organizations replace self-built systems with commercial solutions that spread security expertise across many customers [11]. For many organizations, buying is the correct decision. Building returns dividends only under specific conditions: when data ownership is a hard requirement, when predictable usage-independent cost matters more than delivery speed, when learning-by-building is itself a goal, or when requirements are too unusual for an off-the-shelf product.

G. Gap analysis. The CIAM landscape has settled on a buy-by-default consensus, and for most organizations that is correct. But the same gap keeps surfacing: commercial options are priced per active user, enterprise suites are built for large regulated institutions, and even open-source demands a serious operational commitment. None is shaped for an organization that needs sovereign, predictable, production-grade identity at small scale—where affordability, data ownership, and low operational overhead overlap. TesseractApp addresses that gap.

Table I. The Three Primary Components

Component	Responsibility
Frontend web interface	An Angular 21 single-page application providing users and administrators with registration, in-house and federated login, account management, MFA enrollment, session/device management, and authentication-activity history.
Backend authentication server	A Spring Boot 4 REST API handling all authentication logic, token issuance, role enforcement, and account-lifecycle operations. Spring Security manages both the in-house JWT filter chain and the OAuth2 Client login flow—the centralized identity handler for every incoming request.
Centralized relational database	A MySQL database storing user accounts, BCrypt-hashed passwords, OAuth2 provider identity links, roles and permissions, organization membership, verification tokens, refresh sessions, and audit events.

Table II. Development Phases

Phase	Scope
1: Foundation	Project scaffolding (Spring Boot, Angular, MySQL schema); registration, BCrypt hashing, in-house login; JWT issuance/validation pipeline; stateless Spring Security + CORS.
2: Authorization & accounts	Seven-role permission model; URL- and method-level enforcement; email verification, account enable/disable/lock, role reassignment; password-reset workflow with time-limited email tokens.
3: Hardening & MFA	OAuth2 client for Google/GitHub/Microsoft; token-exchange handler (find-or-create + JWT); authenticator-app (TOTP) MFA and SMS 2FA; silent token refresh with concurrent-refresh protection; token invalidation on password change; anti-enumeration (OWASP OAT-007).
4: Audit & admin	Authentication event logging; admin dashboard for user management and audit review; OWASP Dependency-Check in the build; integration testing and documentation.
5: Threat protection & go-live	Login-anomaly detection with step-up verification and an admin security dashboard; general rate limiting (429 + <i>Retry-After</i>) and per-account brute-force lockout; server-side org-scoped analytics/billing; federated account link/unlink; six-language i18n; a Cmd/Ctrl-K command palette; capability-level UI gating; live deployment on AWS ECS Fargate with a gating CI pipeline.

Table III. The CIAM Landscape, Managed → Self-Reliant

System	Type	Strength	Limitation
Okta / Auth0	Managed SaaS	Developer-first tooling; adaptive MFA; market leader	Per-MAU pricing; large configuration surface; lock-in risk
Microsoft Entra External ID	Cloud SaaS	Deep M365 integration; SSO, SCIM, governance	Best economics only within the Microsoft stack
Ping (+ ForgeRock)	Enterprise	Analyst leader; IGA + MFA + verification	Resource-intensive; enterprise pricing
AWS Cognito	Cloud-native	Native AWS; auto-scales; low cost at scale	Difficult developer experience; sparse documentation; weak edge-case handling
Keycloak (Red Hat)	Self-hosted OSS	Free; full OIDC/SAML; data sovereignty	Operator owns operations, availability, and hardening; steep setup
TesseraApp	In-house	Total control; no per-MAU fee; hybrid model	Full security burden on the implementer

H. Economics. Consumer IAM is estimated at USD 14.12 billion in 2025, rising to USD 22.47 billion by 2030 (9.7% CAGR) [13]; Grand View Research draws the segment boundary differently—USD 8.12 billion in 2023 reaching USD 26.72 billion by 2030 at 17.4% CAGR [14]—but the direction and pace are not in dispute. Standard per-MAU pricing means an organization’s identity expenditure grows with its user base

[11], [16], so the same new users that mark a small firm’s success also raise its identity bill. Small businesses employ nearly half the U.S. workforce and represent 43.5% of GDP [22]; those needing production-grade identity are precisely the segment priced out. Given the USD 4.88 million average breach cost [15], a well-built identity layer also substantially offsets its own cost in risk prevented. The claim is narrow and defensible:

within a specific, underserved segment, building in-house is a sound engineering and economic choice.

III. Project Plan and Schedule

The project was executed over a single academic term in four phases mapped to the four course assignments: the project proposal (motivation and plan), the literature review, the software requirements specification, and the architecture, implementation, and evaluation. Fig. 1 shows the schedule as a Gantt chart; phases overlap where documentation and build proceeded in parallel.

Two features originally scheduled as stretch goals—authenticator-app MFA and refresh-token rotation with reuse detection—were pulled forward and completed within the hardening phase, and are reflected in the chart.

IV. Software Requirements Specification

The specification follows IEEE Std 830-1998. TesseraApp is a self-contained three-tier web application: an Angular 21 single-page application, a Spring Boot 4 / Spring Security 7 REST API and identity server, and a centralized MySQL 8 database. External identity providers, an SMS gateway, and an SMTP service are integrated dependencies.

A. Product perspective. The system provides in-house, centralized control over authentication policy, authorization, and audit logging while allowing users to authenticate with either in-house credentials or a trusted external provider; the two paths converge at a single token-exchange point. Fig. 2 is the system context (level-0) diagram, showing TesseraApp as one process exchanging data flows with its external entities.

B. User classes. Seven roles span three classes—end users (`ROLE_USER`, `ROLE_GUEST`), staff (`ROLE_MODERATOR`, `ROLE_HELP_DESK_ADMIN`), and administrators (`ROLE_ORGANIZATION_ADMIN`, `ROLE_ADMIN`, `ROLE_APPLICATION_ADMIN`). Each user holds exactly one role carrying a permission string; the application admin bypasses organization scope.

B2. Product scope and perspective. TesseraApp provides in-house, centralized control over authentication policy, authorization, and audit logging, while allowing users to authenticate either with in-house credentials or a trusted external identity provider (Google, GitHub, Microsoft); the two paths converge at a single token-exchange point. It is a self-contained three-tier web application: an Angular 21 SPA, a Spring Boot 4 / Spring Security 7 REST API and identity server, and a centralized MySQL database. External identity providers, an SMS gateway (Twilio), and an SMTP service are integrated dependencies. In scope: registration, in-house and federated login, JWT issuance/refresh, MFA, role- and organization-scoped authorization, password lifecycle, audit logging, and an administrative dashboard. Out of scope (this revision): machine-to-machine authorization, SCIM provisioning, and SAML federation.

B3. External interface requirements. User interface: a single Angular SPA with registration, in-house/federated login, verification landings, 2FA entry, password reset, profile with activity history, and an administrative dashboard; dark/light theming persisted across sessions; consistent dismissible error alerts; responsive layout with WCAG 2.1 AA contrast and keyboard navigation on primary flows; runtime six-language i18n (en/es/fr/de/pt/zh) and a Cmd/Ctrl-K command palette.

Software interfaces: OAuth2 Authorization Code + OIDC via Spring Security OAuth2 Client; SMS via Twilio; email via SMTP/JavaMail; MySQL 8 over JDBC; all external credentials injected through environment configuration, never committed to source.

Communications: HTTPS with HSTS (one year, includeSubDomains); RESTful JSON with a consistent response envelope (timestamp, status, code, message, data); bearer access token in the `Authorization` header; restrictive whitelisted CORS; `X-Frame-Options: DENY` and `X-Content-Type-Options: nosniff` on all responses.

B4. Logical data model. Eighteen entities: Users, Roles, User-Role assignment, Organizations plus membership, OAuth provider links, Account / Password-reset / Two-factor verifications, Events plus User-events (the audit trail), TOTP credentials, TOTP recovery codes (SHA-256 hashes only), MFA challenges, and Refresh sessions (user, family, current jti, device, IP, timestamps, revoked/superseded flags). Referential integrity is enforced by foreign keys; single-use keys are unique and expiring; the schema is applied through an idempotent `schema.sql`.

B5. Standards compliance. OAuth 2.0 Authorization Code (RFC 6749) + OIDC Core; JWT (RFC 7519) with HMAC-SHA-512; OWASP Top 10, ASVS for authentication/session management, and OAT-007 anti-enumeration; GDPR/CCPA support via a durable audit trail and minimal federated-data collection; IEEE Std 830-1998 document structure; build-time dependency vulnerability scanning (OWASP Dependency-Check).

C. Functional requirements & use cases. Fig. 3 is the use-case diagram. The End User authenticates (in-house or federated), manages their profile, MFA, and sessions; the Administrator manages users, roles, and reviews audit history; the OAuth2 Provider is a secondary actor to the federated-login use case. Login *includes* the MFA challenge when a second factor is enabled. Table IV, on the following page, enumerates the functional-requirement groups in full.

D. Behavioural model. Fig. 4 is the activity diagram for the authentication flow, the system's most control-heavy path. A first-factor success does not issue tokens directly when MFA is enabled; instead it mints a short-lived challenge, and only a verified second factor reaches the single token-issuance action. All failure paths return a generic error to preserve anti-enumeration.

E. Non-functional requirements. Security: HTTPS + HSTS; BCrypt with salt; short-lived access tokens; server-side au-

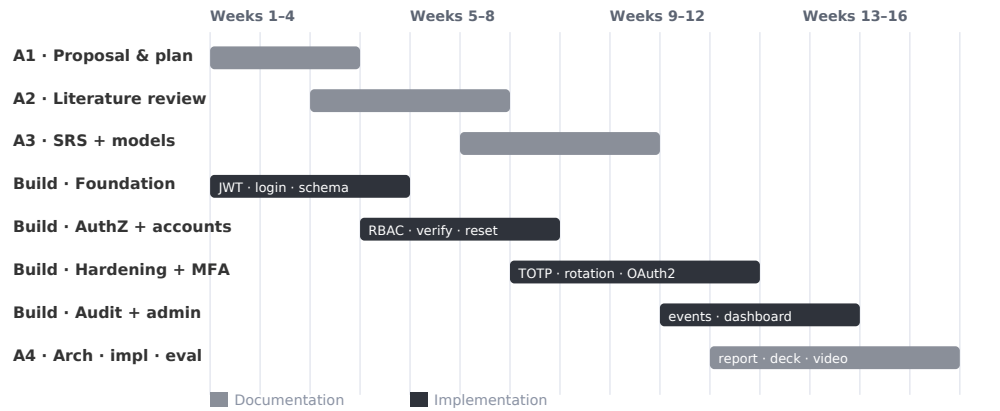


Fig. 1. Project schedule (Gantt). Documentation deliverables (light gray) overlap the four implementation phases (dark gray); MFA and refresh rotation were completed in the hardening phase.

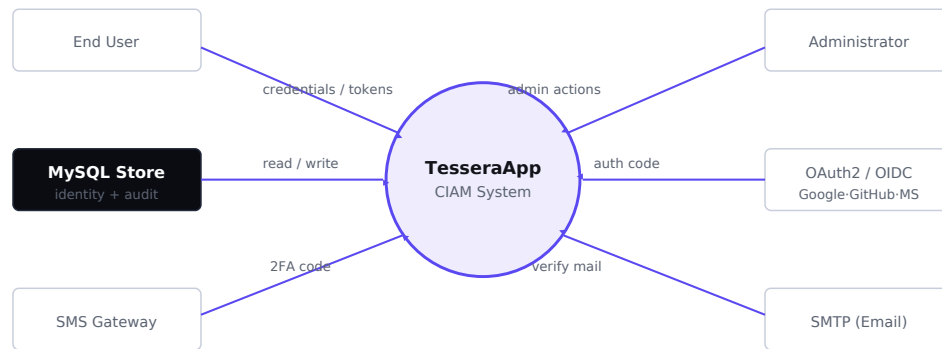


Fig. 2. System context (level-0) diagram. TesserApp mediates every external entity; only the datastore is internal to the trust boundary.

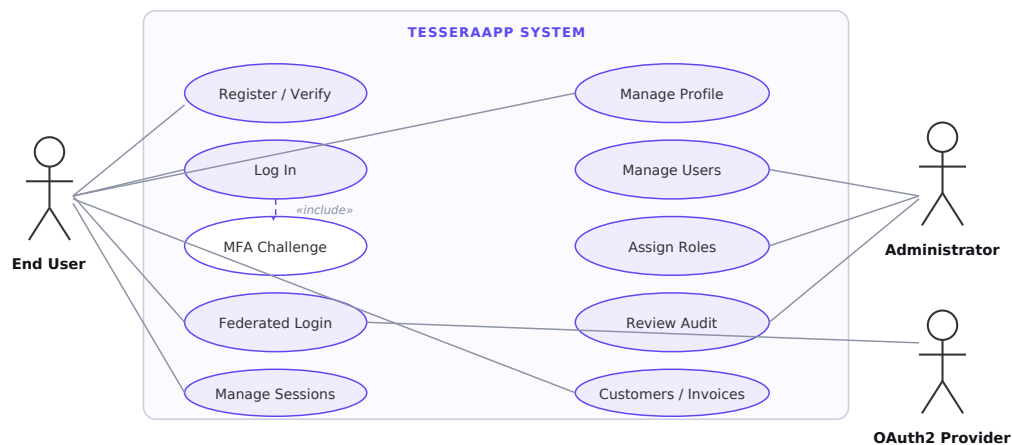


Fig. 3. Use-case diagram. When a second factor is enabled, the Log In use case includes the MFA Challenge (UML *include* relationship); the OAuth2 provider participates as a secondary actor in the Federated Login use case.

thorization on every request; strict anti-enumeration; no self-service privilege escalation. Reliability: deterministic auth decisions; graceful degradation when SMS/email fails. Availability: no session affinity; restarts never force re-

authentication of valid-token holders. Performance: auth request < 500 ms server-side at p95 (excl. external latency); authorization needs no DB lookup in the common case. Portability: runs unchanged on local, Aiven, or AWS RDS MySQL;

Table IV. Functional Requirement Groups (Complete)

Group	Requirements
FR-RAV Registration	Server-side-validated registration; duplicate email rejected without disclosure; only a BCrypt hash stored; account created disabled; a single-use email verification key activates it.
FR-IHA In-house auth	Authenticate via <code>AuthenticationManager</code> + BCrypt; issue tokens on success (unless MFA); generic failure messages; disabled/locked accounts rejected.
FR-FA Federated auth	OAuth2 Authorization Code flow per provider; verify the identity token and extract a stable subject; find-or-create the local user (default <code>ROLE_USER</code>); issue our own JWT so RBAC/MFA/audit apply identically; persist only provider name + subject id—never a third-party password. Signed-in users link/unlink providers via a single-use, provider-bound ticket; linking refuses an identity owned by another account, and the last remaining sign-in method cannot be unlinked.
FR-JWT Tokens	30-min access + 5-day refresh, HMAC-SHA-512; authorities embedded in the access token; a stateless filter validates every request; interceptor with single-flight refresh; every refresh recorded as a server-side session (jti, device, IP); reuse detection revokes the whole family and audits <code>TOKEN_REUSE_DETECTED</code> ; password change invalidates all prior tokens via <code>passwordChangedAt</code> .
FR-MFA Multi-factor	Enable/disable from the profile; SMS one-time code withholds tokens until verified; authenticator-app (TOTP) offered as an alternative, taking precedence over SMS; single-use SHA-256-hashed recovery codes.
FR-RBAC / FR-ORG Authorization	One role per user; permission strings become Spring authorities enforced at URL <i>and</i> method level; 401 via entry point, 403 via access-denied handler; no self-elevation; a 1–7 privilege-tier ladder bounds role assignment; org admins scoped to their own organization; the application admin bypasses scope.
FR-PM Passwords	Neutral reset response regardless of email existence; single-use, time-limited reset key by email; no key or password in a query string; signed-in change reissues tokens; a centralized complexity policy (minimum 8 characters, upper- and lowercase letters, and a digit) enforced identically at registration, change, and reset; BCrypt with per-password salt.

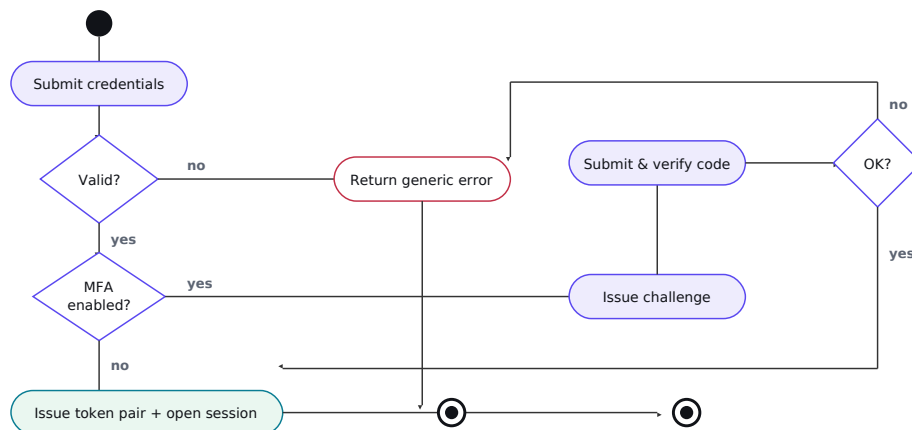


Fig. 4. Activity diagram — login with optional MFA. Every path either reaches the single token-issuance action or returns a generic error.

all environment values externalized. Table V, on the following page, gives the full set.

Table IV. Functional Requirement Groups (Complete) (cont.)

Group	Requirements
FR-AEL Audit	Record login success/failure, federated login, password/profile/role/settings/MFA/image changes across 18 event types; capture type, method, timestamp, IP, device; events for known accounts only (no enumeration); user-visible paginated history; immutable.
FR-AUM Administration	Paginated searchable directory within scope; single-user detail with role, state, and event history; scoped role reassignment and account-state changes, each audited; administrative session revocation; every admin endpoint behind an explicit authority.
FR-BM Business domain	CRUD customers with pagination + search; invoices with line items, status, amounts; a services/billing catalog with staff-only administration; profile-image upload on portable object storage.
FR-SDM Sessions & devices	View active sessions (device, IP, sign-in, last activity, expiry, current highlighted); revoke one; revoke all others; revocations audited.
FR-TPF Threat protection	Login-anomaly detection scores each sign-in on device/network novelty and escalates an unrecognized context to step-up verification; a general rate limiter returns 429 + <code>Retry-After</code> alongside the per-account brute-force lockout; an admin security dashboard surfaces anomalous sign-ins, authentication trends, restricted accounts, and MFA adoption from the audit trail.
FR-PWL Passkeys	Discoverable WebAuthn registration and sign-in via <code>webauthn4j-core</code> ; usernameless login resolves the account from the assertion's credential id; a successful passkey login bypasses step-up/risk assessment; self-service and administrative revocation only—no reset path.
FR-UX Experience	Runtime six-language i18n with instant switching; a <code>Cmd/Ctrl-K</code> command palette over every navigable destination; capability-level UI gating so controls a user cannot exercise are disabled with an explanation rather than failing on submit.

Table V. Quality Attributes (Non-Functional Requirements)

Quality	Key requirements
Security (NFR-SEC)	HTTPS + HSTS; BCrypt with per-password salt; short-lived access tokens; server-side authorization on every request (route guards are usability only); CSRF disabled for the stateless API, HTTP Basic disabled, whitelisted CORS; anti-clickjacking and MIME-sniffing headers; strict anti-enumeration—no message, code, or timing difference reveals account existence; no self-service privilege escalation; minimal federated data retained.
Reliability (NFR-REL)	Deterministic authentication decisions; graceful degradation when SMS or email delivery fails; multi-row writes never leave an account unusable.
Availability (NFR-AVAIL)	No session affinity—any instance serves any request; container health check; restarts never force re-authentication of valid-token holders.
Performance (NFR-PERF)	Auth request < 500 ms server-side at p95 (excluding external-provider latency); authorization requires no database lookup in the common case; all lists and audit views paginated.
Portability (NFR-PORT)	Runs unchanged on local, Aiven, or AWS RDS MySQL; any Java 21 + container platform; all environment values externalized.
Maintainability (NFR-MAINT)	Independent, isolable modules; current framework idioms; idempotent <code>schema.sql</code> ; documentation that explains purpose rather than restating signatures.

V. System Architecture and Design

The system uses the 4+1 view model. Three tiers—Angular SPA, Spring Boot REST API, MySQL—communicate over JSON and JDBC. The API is stateless; the only intentionally

stateful element is the refresh-session store, which restores revocability to an otherwise stateless token model.

A. Views. The *logical* view splits into Identity & Authentication, Access Control, and Business domains

plus a cross-cutting Audit module, layered Controller→Service→Repository. The *process* view covers per-request handling: login → optional challenge → token issuance, and refresh → rotation with reuse detection. The *development* view is the layered backend packages and standalone-component Angular. The *physical* view runs native dev processes or a containerized app + DB, and is deployed in production on AWS ECS Fargate (ECR, ALB, Secrets Manager, CloudWatch) against Aiven managed MySQL.

B. The hybrid token model. Access is stateless: a 30-minute HMAC-SHA-512 JWT carries the user’s authorities and is verified by signature alone, with no database hit on the hot path. Refresh is stateful: every refresh token is a tracked row (family + jti + device + IP), so sessions can be listed, rotated, and revoked. On refresh, the old row is superseded and a new jti inserted; a replayed superseded token is treated as theft, revoking the whole family and auditing `TOKEN_REUSE_DETECTED`.

A critical design decision. The rotation routine is deliberately not transactional: the family revocation must commit *before* the rejection is thrown, or the throw would roll the revocation back—reversing the very containment it had just performed. Writes are ordered supersede-then-insert, so a crash mid-operation leaves an already-retired token, never two live tokens in one family.

C. Authentication convergence. Password login, TOTP-verified login, and federated OAuth2 login all end at one seam—the session service that mints the application’s own JWT and opens a tracked session—so RBAC, MFA policy, and audit apply identically regardless of the authentication path used. Federated logins even appear in the Security Center device list exactly like in-house ones.

D. Entity-relationship model. Fig. 6 is the entity-relationship diagram for the core relational schema. The `users` table is the identity hub: one-to-many to `userroles` (role assignment), `usevents` (device- and IP-stamped audit rows), and the single-use verification tables—`accountverifications`, `resetpasswordverifications`, and `twofactorverifications`—each keyed by a unique, expiring token. The business domain forms a separate cluster: `customer` → `invoice` → `invoiceserviceitems`, priced against a `services` catalogue. Referential integrity is enforced by foreign keys throughout. The Phase-2 identity tables (TOTP credentials, recovery-code hashes, MFA challenges, and refresh sessions) extend this core and are catalogued in Section IV-B4.

VI. Implementation and Interface

The frontend is Angular 21 / Bootstrap 5 / RxJS; the backend is Spring Boot 4 / Java 21 / Spring Security 7. Identity data uses hand-written `NamedParameterJdbcTemplate` SQL so every query touching identity is explicit and auditable; the CRUD-heavy business domain uses JPA. Federation uses Spring Security OAuth2 Client; ZXing rasterizes TOTP QR codes; Apache POI exports XLSX; a multi-stage Docker build packages the SPA into the Spring Boot JAR.

A. Security seams. `SecurityConfig` defines the stateless filter chain; `CustomAuthFilter` validates the bearer token per request and authenticates only tokens carrying authorities; `TokenProvider` mints/verifies JWTs and enforces `passwordChangedAt`; `SessionServiceImpl` is the single issuance seam (issue + rotate with reuse detection); `TotpServiceImpl` handles challenge-bound TOTP and recovery codes; `OAuth2LoginSuccessHandler` is the federated token-exchange point.

B. Passkeys (WebAuthn). `TesseraApp` adds discoverable, passwordless authentication via WebAuthn, built directly on `webauthn4j-core` rather than Spring Security’s session-based WebAuthn module—the application is stateless JWT throughout, and that module assumes an `HttpSession` in which to hold ceremony state. A purpose-built `WebAuthnChallengeStore` (in-memory, single-use, five-minute TTL) fills that role instead, structurally identical to the OAuth account-linking ticket store already in use elsewhere. Login is *usernameless*: no identifier is submitted; the browser’s own credential picker surfaces any passkey registered for the origin, and the server resolves the account solely from the credential id returned in the assertion. Because a passkey is already phishing-resistant and bound to one authenticator, a successful passkey login bypasses login-risk assessment and step-up entirely and mints tokens directly—the same treatment already given to federated OAuth2 logins (Section VI-A). Revocation, self-service or administrative, is the only recovery path: the private key never leaves the authenticator, so there is nothing to reset. Enrollment itself is nudged rather than mandated: an account with no registered passkey is shown a one-time, dismissible prompt immediately after its next successful login, offering enrollment on the spot or deferral to Security Center; the prompt does not resurface once dismissed.

C. Two-phase delivery. Phase 1 stood up the full stack and demonstrated registration + verification, in-house login, JWT access/refresh, seven-role RBAC, audit logging, and the customer/invoice domain. Phase 2 completed authenticator-app MFA with recovery codes, refresh-session rotation with reuse detection and a user-facing Security Center, OAuth2 federated login with account link/unlink, and organization-scoped administration with a privilege-tier ladder (blocking privilege-elevation-by-proxy) and administrative session revocation; added a full threat-protection layer (login-anomaly detection with step-up, an admin security dashboard, a two-tier rate limiter, and per-account lockout), a capability catalog for specific 403s, six-language i18n, and a command palette; consolidated schema management into an idempotent script; and deployed the system to production on AWS ECS Fargate (ECR, ALB, Secrets Manager, CloudWatch) against Aiven managed MySQL, packaged by a multi-stage Docker build.

D. User interface. A dark-first “security console” theme over Bootstrap 5: deep space-blue surfaces, an electric-iris accent used only for primary actions and focus, glass cards, and IBM Plex Sans/Mono type. Every async view has explicit loading/loaded/error states; feedback is non-blocking toast. Phase



Fig. 5. Three-tier architecture; only the refresh-session store is stateful.

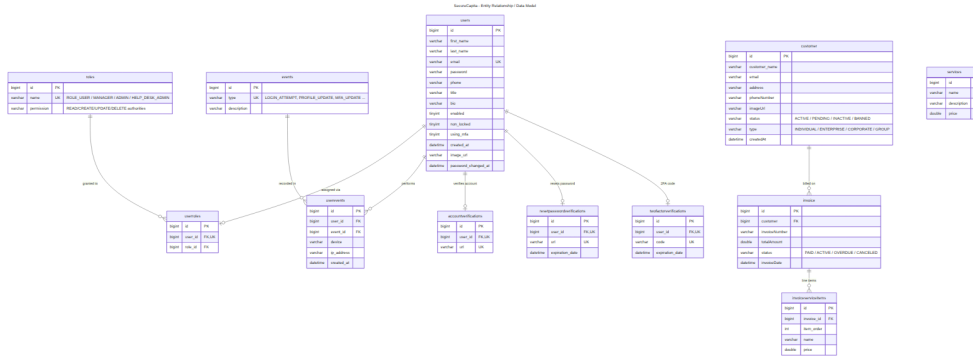


Fig. 6. Entity-relationship / data model. The `users` table anchors identity; the customer–invoice–line-item chain forms the business domain; audit, role, and verification tables all key back to `users`.

ID	PHOTO	NAME	EMAIL	ROLE	RFA	STATUS	ACTION
32		Bob AWS	bobaws@gmail.com	ROLE_APPLICATION_ADMIN	OFF	ACTIVE	Manage
31		Robert Oliver	robert_ryff@outlook.com	ROLE_USER	OFF	ACTIVE	Manage
30		Bob Test	bobtest@gmail.com	ROLE_APPLICATION_ADMIN	ON	ACTIVE	Manage
29		Robert Oliver	robert_ryff@yahoo.com	ROLE_USER	OFF	ACTIVE	Manage
28		Robert Oliver	robertoliver@lewisu.edu	ROLE_USER	OFF	ACTIVE	Manage
27		Robert Oliver Jr	robertoljr12@gmail.com	ROLE_USER	OFF	ACTIVE	Manage
26		Bob Oliver	bob11@gmail.com	ROLE_USER	OFF	ACTIVE	Manage
25		Frank AppAdmin	frank.app@tessera.dev	ROLE_APPLICATION_ADMIN	OFF	ACTIVE	Manage
24		Dave OrgAdmin	dave.org@tessera.dev	ROLE_APPLICATION_ADMIN	OFF	ACTIVE	Manage
23		Bob Mod	bob.mod@tessera.dev	ROLE_MODERATOR	OFF	ACTIVE	Manage

Fig. 7. Organization-scoped admin user directory—searchable, paginated, and enforced server-side by the security seams of Section VI-A.

2’s staff-only surfaces—the user directory (Fig. 7), the security overview (Fig. 8), analytics and billing hubs, and services-catalog administration—are reached through *Admin* and *Insights* menus rendered only for staff-grade accounts; route guards and the `/admin/**` rules enforce the same authorities regardless. The console ships with dark and light themes behind a persisted one-click toggle; Figs. 7–8 show the light theme.

E. Security seams in detail. Table VI enumerates the critical classes and the property each protects.

F. Current status. The advanced security set (TOTP, refresh rotation, sessions, federated login/link, anomaly detection, se-

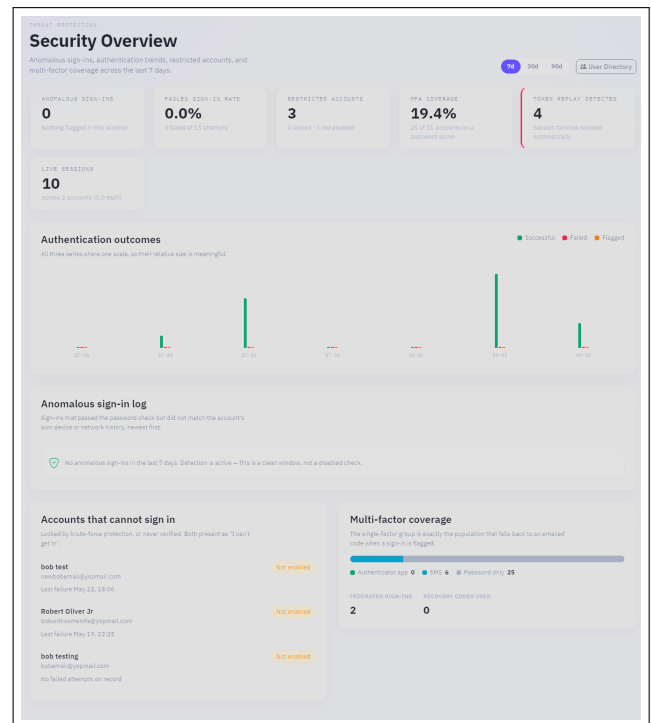


Fig. 8. Admin security overview (FR-TPF-2)—anomalous sign-ins, failed-attempt rate, restricted accounts, MFA coverage, and token-replay revocations over a selectable window.

curity dashboard), the rate limiter, server-side analytics/billing gating, six-language i18n, and the CI-gated 164/87 test suite are all implemented, and the system is live on AWS ECS Fargate behind a CloudFront distribution that serves HTTPS, reachable at the project’s own domain (<https://tesseraapp.dev>,

Table VI. Critical Security Classes

Class	Responsibility
SecurityConfig	Filter chain: stateless sessions, CORS, ordered authority rules, OAuth2 login, custom 401/403 handlers, security headers.
CustomAuthFilter	Per-request bearer validation; sets the security context only for tokens carrying authorities—a refresh token can never authenticate an API call.
TokenProvider	Mints/verifies HMAC-512 JWTs; enforces the <code>passwordChangedAt</code> check that makes a password change invalidate every outstanding token.
SessionService Impl	The single token-issuance seam—issue + rotate with reuse detection (commit-then-throw); lists and revokes sessions.
TotpServiceImpl	Authenticator-app lifecycle: enroll, confirm, challenge-bound verify, single-use recovery codes (SHA-256 hashed).
OAuth2Login SuccessHandler	The federated token-exchange point: per-provider profile extraction, find-or-create, MFA parity, then issues our own JWT.
PasskeyService Impl	WebAuthn registration and assertion verification via <code>webauthn4j-core</code> ; mints/redeems challenges through <code>WebAuthnChallengeStore</code> ; resolves the account from the credential id on login—the only identity signal a passkey login carries.

ROLE	ROLE_ADMIN	ROLE_ORG_ADMIN	ROLE_APP_ADMIN	ROLE_ORG_READ	ROLE_APP_READ	ROLE_ORG_WRITE	ROLE_APP_WRITE	ROLE_ORG_DELETE	ROLE_APP_DELETE
ROLE_USER	-	-	-	-	-	-	-	-	-
ROLE_ORG_ADMIN	-	-	-	-	-	-	-	-	-
ROLE_APP_ADMIN	-	-	-	-	-	-	-	-	-
ROLE_ORG_READ	-	-	-	-	-	-	-	-	-
ROLE_APP_READ	-	-	-	-	-	-	-	-	-
ROLE_ORG_WRITE	-	-	-	-	-	-	-	-	-
ROLE_APP_WRITE	-	-	-	-	-	-	-	-	-
ROLE_ORG_DELETE	-	-	-	-	-	-	-	-	-
ROLE_APP_DELETE	-	-	-	-	-	-	-	-	-

Fig. 9. The role-to-permission matrix (dark theme)—each column is a permission string enforced by Spring Security at URL and method level.

Fig. 10. The access console—credentials branch into an MFA challenge when a second factor is enabled; all three federated providers are configured.

DNS-aliased to the CloudFront distribution with an ACM-issued certificate) on AWS’s own certificate infrastructure, so TLS, HSTS, and secure-context features are available under a real domain rather than the original `*.cloudfront.net` hostname (the ALB behind CloudFront still serves plain HTTP). All three federated providers (Google, GitHub, Microsoft) are now live in the deployed environment with real credentials and registered callbacks pointed at the new domain. The database link is now fully hardened—every deployed profile requires TLS with full certificate-identity verification (`sslMode=VERIFY_IDENTITY`) against a pinned Aiven CA. Remaining items for a hardened production posture: moving per-instance security state (brute-force counters, rate-limit buckets, link tickets) off the JVM heap before running a second instance; a rehearsed backup/restore with automated schema application; and SMS 2FA, which is Twilio-credentialed and code-complete but still awaiting Twilio’s A2P 10DLC campaign verification, so it currently degrades to a logged code.

G. Phase 2 by the numbers. Table VII measures the delta between the Phase 1 baseline (June 19) and the final head, comparing app-relative paths across the frontend workspace rename so only genuine additions count.

The 43 new backend classes group into five capabilities: risk-based login and step-up (10—`LoginRiskServiceImpl` and its context, query, and repository types); the security dashboard (15—controller, service, repository, and the aggregate models behind its panels); analytics and the services catalog (4—`AnalyticsController` closed a real authorization gap where billing figures had been reachable through a customer-scoped endpoint with only client-side gating); object storage (4—`ImageStorageService` behind local and S3 implementations); and hardening infrastructure (10—rate limiting, trusted-proxy handling, `AuthDiagnosticsLogger` classifying every denial into seven server-side reasons while the client response stays unchanged, error-detail scrubbing, the capability catalog, password pol-

Table VII. Phase 2, Measured

Measure	Value
New backend classes	43
New backend test classes	22 (27 suites total)
New frontend source files	18 (+ 8 spec files)
Frontend files reworked	103 of 136
Locales	6 × 495 keys, exact parity (machine-checked)
Automated tests	14 → 251 (164 backend / 87 frontend) · ×17.9

icy, provider-link tickets, and serialization/template configuration). New endpoint groups: `/admin/analytics/**`, `/admin/security/**`, `/admin/services/**`, and `DELETE /admin/user/{id}/sessions`.

H. CI/CD and environments. The CI workflow runs `mvn verify` with OWASP Dependency-Check (failing on $CVSS \geq 7$), `npm audit`, a gating `ng lint`, the frontend suite, and a production build against a live MySQL service container; the deploy workflow ships to ECR and ECS Fargate only after the full CI workflow passes, and a parallel pipeline targets GCP Artifact Registry and Cloud Run (built, not deployed). Four environment profiles—dev, qa, stage, prod—drive a single env-configured image; production pins `ddl-auto: validate`. Operational logs go to CloudWatch (7-day retention, env-driven levels) in the AWS deployment and to standard console logging locally.

I. Roadmap scorecard. Phase 1 committed to eight follow-on items; seven were delivered—security-critical-path tests, frontend specs, MySQL port exposure, dependency cleanup, prod-profile validation, object storage, general rate limiting—and one was not: `start.sh` still calls the system `mvn` rather than `./mvnw`, a one-line fix. Of Phase 1’s seven carried-forward limitations, six are closed; only the example environment file still shipping a placeholder JWT secret remains, mitigated by a guard that fails fast under the production profile.

J. Defects found and fixed. A 20-entry problem log accompanies the implementation record; each entry was identified during real use rather than during review. Table VIII lists the most instructive, several of which now have a regression test standing guard.

Table VIII. Defect Log (Highlights)

Defect	Resolution
Login returned 500 for every user	A schema drift made audit inserts fail and the synchronous event multicaster propagated the failure into login; the listener now swallows and logs, so a failing audit write can never break authentication (guarded by <code>NewUserEventListenerTest</code>)
Analytics authorization gap	Billing/analytics data was gated only client-side → dedicated <code>AnalyticsController</code> with <code>@PreAuthorize</code>
Two CORS policies silently disagreed	A hardcoded list beat the configurable one; unified into one bean
Admins saw the non-admin view after token refresh	Authority flags were latched at construction rather than read from the current token
A usable bearer token was logged to the browser console	Removed
Table-name casing differed between environments	Every query aligned to <code>schema.sql</code> ; guarded by <code>SqlTableCaseConsistencyTest</code>
CSP broke icons and fonts in production only	Policy corrected
A rejected JWT returned 400, so silent refresh never fired	Corrected to 401
<code>devMessage</code> leaked in production	<code>ErrorDetailScrubber</code> strips it
Seeded role ids drifted between databases	Privilege tiers moved into code, deliberately not read from the database
N+1 queries on customer/invoice lists	Fixed with <code>@BatchSize(50)</code> on the invoice/line-item collections—batched <code>IN</code> loads, ~96% fewer SQL round-trips

VII. Testing and Evaluation

Security tooling includes OWASP Dependency-Check in the Maven build (failing on $CVSS \geq 7$), ESLint/Prettier on the frontend, and a build-breaking guard on a weak or missing JWT secret. An automated suite of 164 backend and 87 frontend tests gates CI—covering refresh rotation and reuse detection, login-enumeration parity, brute-force lockout, login-risk assessment, security-dashboard scoping, organization scope, federated link/unlink, TOTP challenge binding, and the token interceptor’s single-flight refresh. All primary flows were also verified end-to-end against a live database (evidenced by the demo videos): register → verify → login → MFA → dashboard; admin role reassignment with audit; and the password-change-invalidates-all-tokens guarantee. The remaining coverage gap is browser-level end-to-end (Playwright) testing over the seams; no performance benchmarks are claimed.

VIII. Conclusion and Future Work

We built a complete, modern CIAM core on a real application, validating a hybrid stateless/stateful authentication model that balances scalability with revocability. For most organizations, buying identity is the correct decision; but inside a specific, underserved window—schools, startups, small businesses, and non-profits that need sovereign, predictable, production-grade identity at small scale—building in-house is a defensible engineering and economic choice, and TesseractApp shows it can be done correctly.

With the system now live on AWS and the SRS feature

set complete, future work is quality and operability rather than features: browser-level end-to-end (Playwright) tests in CI; hardening the JDBC connection to full certificate-identity verification—TLS is already required via `sslMode=REQUIRED` on the deployed profiles; moving the per-instance security state (rate-limit buckets, brute-force counter, link tickets) to a shared store for horizontal scale-out; and checking submitted passwords against a breach corpus (e.g., Have I Been Pwned’s k-anonymity API) on top of the existing complexity policy—deferred because it adds an outbound network dependency to the registration path. Further out: shared-schema multi-tenancy for the business domain.

Author Contributions. R. C. Oliver, Jr. was responsible for the system architecture, the full backend and frontend implementation, all deployment and CI/CD engineering, and authored all six graded project deliverables (proposal/SRS, Implementation 1, Implementation 2, this Final Report, and the accompanying presentation materials), as well as all presentation slides and both recorded demonstration videos. R. C. Oliver, Jr. wrote the great majority of the project’s code end to end. T. Lester contributed to brainstorming the project’s direction, assisted with early thinking on the AWS deployment approach and RBAC role design, application advantages and potential constraints, contributed to select written sections and the project’s demo and presentation materials, helped edit the first and second presentation/demo videos, and helped proofread submissions.

Acknowledgment. We thank Dr. Al-Sharif for advising this project, and Junior of GetArrays (Udemy, YouTube), whose

full-stack Angular and Spring Boot course inspired the baseline application from which this project grew. In accordance with Lewis University's Academic Integrity policy, we disclose that AI tools assisted in the preparation of the project's deliverables; all architectural decisions, design choices, and analysis are our own.

References

- [1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST SP 800-207, Aug. 2020.
- [2] N. F. Syed *et al.*, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022.
- [3] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. ACM CCS*, 2016, pp. 1204–1215.
- [4] T. Baumer, S. Kern, L. Fuchs, and G. Pernul, "Identity and access management metrics: A systematic review," *ACM Comput. Surv.*, vol. 58, no. 8, 2026.
- [5] S. Simpson and T. Groß, "A survey of security analysis in federated identity management," in *Privacy and Identity Management*, Springer, 2016, pp. 231–247.
- [6] M. Kepkowski *et al.*, "Challenges with passwordless FIDO2 in an enterprise setting: A usability study," in *Proc. IEEE SecDev*, 2023, pp. 37–48.
- [7] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," IETF RFC 6749, Oct. 2012.
- [8] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.
- [9] D. M'Raihi *et al.*, "TOTP: Time-Based One-Time Password Algorithm," IETF RFC 6238, May 2011.
- [10] N. Sakimura *et al.*, "OpenID Connect Core 1.0," OpenID Foundation, Nov. 2014.
- [11] B. Guthrie *et al.*, "Magic Quadrant for Access Management," Gartner, Dec. 2024.
- [12] J. Tolbert, "Leadership Compass: CIAM," Kuppinger-Cole, LC80904, Jan. 2026.
- [13] MarketsandMarkets, "Consumer IAM (CIAM) Market — Global Forecast to 2030," 2025.
- [14] Grand View Research, "Customer IAM Market Size, Share & Trends Report, 2030," 2024.
- [15] IBM Security, "Cost of a Data Breach Report 2024," IBM Corp., 2024.
- [16] Auth0 by Okta, "Customer Identity and Access Management (CIAM)." [Online].
- [17] Microsoft, "Microsoft Entra External ID documentation." [Online].
- [18] Ping Identity, "Product documentation." [Online].
- [19] Amazon Web Services, "Amazon Cognito Developer Guide." [Online].
- [20] Keycloak, "Keycloak Documentation." [Online].
- [21] IAM Experts, "The Evolution of IAM, A Look Back and Beyond." [Online].
- [22] U.S. Chamber of Commerce, "Small Business Data Center." [Online].

Appendix

A. Glossary (selected). CIAM—customer identity & access management, for external self-service users. JWT—a signed, self-contained token carrying identity and authority claims. Access token—the short-lived JWT presented on every request. Refresh token—the longer-lived, server-tracked, rotatable token that mints new access tokens. Reuse detection—treating a consumed refresh token presented again as theft, revoking the whole session family. Token-exchange point—the single place any successful login, in-house or federated, becomes the application's own JWT. Enumeration—discovering which accounts exist from response differences; designed against throughout.

B. Demo accounts. Seeded with password TesseractDemo@1: alice.guest (GUEST), bob.mod (MODERATOR), carol.help (HELP_DESK_ADMIN), dave.org (ORGANIZATION_ADMIN), eve.admin (ADMIN), frank.app (APPLICATION_ADMIN)—all @tesseract.dev.

C. API summary. Endpoint groups: /user/** (auth, profile, MFA, sessions), /oauth2/** + /login/oauth2/** (federation and link/unlink), /totp/** (authenticator), /session/** (devices), /admin/user/** (administration), /admin/analytics/** and /admin/security/** (org-scoped analytics and the security dashboard), /admin/services/** (catalog administration), /customer/** (customers, invoices, services).

D. Requirements traceability. Each identifier (FR-, NFR-, EIR-, DB-, DC-, SC-, OR-*) traces forward to a design element, a code module, and a test case; 36 distinct requirement IDs are cited at over 400 sites in the source itself, so traceability is verifiable by search rather than by assertion. Phase mapping: Phase 1 → FR-RAV, FR-IHA, FR-JWT; Phase 2 → FR-RBAC, FR-PM, FR-AEL; Phase 3 → FR-FA, FR-MFA, FR-AUM; Phase 4 → FR-BM, FR-SDM, FR-TPF; Phase 5 → FR-TPF (anomaly + dashboard), FR-UX (i18n, command palette, capability gating), and production go-live.

E. Companion documents. The System Architecture & UI Design report (full 4+1 treatment and diagrams); the Implementation 1 and Implementation 2 reports; the Final Presentation deck with its presentation & demo guide and speaker

notes; and `final_implementation_guide.md`, the remaining-work development guide.